System Design Document

Document History

- Version: 1.0
- Date: March 7, 2026
- Author(s): Group 1

---

1. Introduction

1.1 Purpose

The purpose of this document is to provide a comprehensive System Design overview for the **SigmaTutor** platform. It details the high-level architecture, component design, database schemas, and interface structures to guide development, maintenance, and future scalability.

1.2 Scope

This document covers the entire SigmaTutor ecosystem, which includes a user-facing Next.js web application (Frontend), an Express.js REST API (Backend), and a PostgreSQL database managed via Prisma ORM.

1.3 Referenced Documents

- `package.json` configurations (root, frontend, backend)
- Database schema definitions (Prisma)
- API Route implementations

---

2. System Overview

SigmaTutor is an online and onsite tutoring and course management platform. The system allows users to browse courses, enroll, make payments, and access learning materials (chapters and lessons). Instructors and administrators can manage course content, schedules, and platform settings.

2.1 Main Features

- **User Authentication**: JWT-based login, Google OAuth integration.
- **Course Management**: CRUD operations for courses, chapters, levels, and categories.
- **Enrollment & Payments**: Stripe integration for handling course purchases.
- **Scheduling**: Management of onsite or live-online course schedules.
- **Content Delivery**: AWS S3 integration for storing and serving course media/files.
- **User Roles**: Support for students, tutors, and administrators.

2.2 Architecture Style

The system follows a **Client-Server Architecture** deployed as a Monorepo using Turborepo. It features a decoupled frontend (Next.js SSR/CSR) communicating with a monolithic Node.js backend (Express) via RESTful API endpoints.

---

3. Design Constraints

3.1 Technological Dependencies

- **Operating System**: Cross-platform (containerized via Docker).
- **Frontend Framework**: Next.js 16+, React 19, Tailwind CSS 4.
- **Backend Framework**: Express.js 5.x on Node.js (>= 18.0.0).
- **Database**: PostgreSQL (via Prisma ORM), Redis (for caching and rate limits).
- **Package Manager**: pnpm.

3.2 Performance & Security Requirements

- **Caching & Rate Limiting**: Redis is used to limit API request rates (`express-rate-limit` , `rate-limit-redis`) protecting against brute-force and DDoS.
- **Data Protection**: `bcryptjs` for password hashing, `helmet` for HTTP headers security.
- **Authentication**: Stateless JWT mechanism; secure cookie parsing.
- **File Uploads**: `multer` handling multipart/form-data before uploading to AWS S3.

---

4. Design Assumptions

4.1 User Behaviors

- Users will primarily interact with the system via modern, JavaScript-enabled web browsers.
- Administrative users require dedicated, protected interfaces with elevated access controls.

4.2 External API Dependencies

- **Stripe**: Used exclusively for payment processing. Assumes continuous availability of Stripe's API.
- **AWS S3**: Used for reliable cloud object storage.
- **Google OAuth**: Relies on Google's Identity services for social login routing.

---

5. System Architecture

5.1 High-Level Architecture Diagram

```
+-------------------+       +-----------------------+       +-------------------+
|                   |       |                       |       |                   |
|   Web Browser     |<=====>|   Next.js Frontend    |<=====>|   Express.js API  |
|   (Client)        | HTTP  |   (SSR / React)       | REST  |   (Backend)       |
|                   |       |                       |       |                   |
+-------------------+       +-----------------------+       +----------+--------+
                                                                        |
                                                                        |
                  +-------------------+       +-------------------------+--------+
                  |                   |       |                                  |
                  | External Services |       |          Data Persistence        |
                  | - Stripe API      |<=====>| - PostgreSQL (Prisma ORM)        |
                  | - AWS S3          |       | - Redis (Cache/Rate Limit)       |
                  | - Google OAuth    |       |                                  |
                  +-------------------+       +----------------------------------+
```

5.2 Main Components

1. **Frontend App**: Server-side rendered React application responsible for UI, routing, and client-side state management (via TanStack React Query).
2. **Backend API**: Serves JSON data, orchestrates business logic, handles background jobs, and interacts with third-party providers.
3. **Database Layer**: Centralized PostgreSQL data store handling relational integrity, structured with Prisma.
4. **Cache Layer**: Redis cluster handling transient data and session rate-limiting.

---

6. Component Design

6.1 Authentication Module (`auth.routes.ts` , `user.routes.ts`)

- **Responsibility**: Handle user registration, login, token generation, and OAuth callbacks.
- **Flow**: User submits credentials -> Backend hashes/compares with DB -> Issues JWT -> Returns to client.

6.2 Course Module (`course.routes.ts` , `chapter.routes.ts` , `lesson.routes.ts`)

- **Responsibility**: Serve nested content structures (Course -> Category/Level -> Chapter -> Lesson).
- **Inputs**: Course IDs, filter parameters.
- **Outputs**: JSON arrays of course structures, video links, or document metadata.

6.3 Payment Module (`payment.routes.ts`)

- **Responsibility**: Initiate Stripe checkout sessions and handle webhooks for successful transactions.
- **Flow**: User clicks "Buy" -> Backend creates Stripe Session -> Client redirects to Stripe -> Webhook confirms payment -> Backend creates User Enrollment record.

---

7. Database Design

7.1 Schema Overview

The database uses a relational model managed by Prisma. The primary entities and their relationships include:

- **User**: Stores authentication details, roles, and profile information.
  - *Relations*: 1-to-Many with Reviews, Enrollments, and Schedules (as an instructor).
- **Course**: The core product offering.
  - *Relations*: Belongs to a Category and a Level. 1-to-Many with Chapters and Reviews.
- **Chapter & Lesson**: Represents the hierarchical structure of a course. A Course has many Chapters; a Chapter has many Lessons.
- **Schedule**: Manages temporal data for live or onsite classes. Linked to specific Courses and Instructors.
- **Coupon & Promotion**: Financial modifiers for checkout.
- **Review**: User feedback linked to specific Courses.

7.2 Data Flow Example (Course Purchase)

1. `User` selects `Course` .
2. System checks `Promotion` and `Coupon` tables for discounts.
3. Upon Stripe success, an `Enrollment` (implicit or explicit join table) links `User` and `Course` .

---

8. Interface Design

8.1 Public Interfaces (`app/(public)`)

- `/explore` : Course catalog with search, filtering, and pagination.
- **Home/Landing**: Marketing page showcasing banners, promotions, and popular courses.

8.2 Authentication Interfaces (`app/(auth)`)

- **Login/Register**: Forms for credential input. Form state managed by `react-hook-form` .

8.3 Main Application Interfaces (`app/(main)`)

- **Admin Dashboard (`/admin`)**: Protected routes for content generation.
  - `/admin/courses/create` : Form interface for adding new courses, chapters, and lessons.
  - `/admin/marketing` : Interfaces for updating homepage banners and promotional content.
- **Student Dashboard**: Interface for viewing enrolled courses, tracking progress, and communicating with tutors.
- **Component Library**: Includes reused UI elements like Course Cards, Video Players, and Modals implemented with Tailwind CSS and Radix/Lucide icons.